

Technical Documentation

Personal Safety & Emergency SOS Platform – Mobile App and Backend API

Version 1.0 · October 2026
Repository: unlikelycreator/Jymberee

Contents

1. Introduction 1
- 1.1 Purpose of this document 1
- 1.2 Audience 1
- 1.3 Scope and source 1

2. Product overview 2
- 2.1 What Jymberee does 2
- 2.2 Core features 2
- 2.3 SOS triggers by platform 3

3. System architecture 4
- 3.1 High-level view 4
- 3.2 Technology stack 4
- 3.3 External services 5
- 3.4 Environments 5

4. Repository structure 6

5. Backend 7
- 5.1 Startup sequence 7
- 5.2 Request pipeline (app.js) 7
- 5.3 API versioning 7
- 5.4 Authentication 7
- 5.5 Validation and errors 8
- 5.6 Environment variables 8
- 5.7 Running locally 8

6. Data model 9
- 6.1 User 9
- 6.2 Other collections 10

7. API reference (v3) 11
- 7.1 Auth – /auth 11
- 7.2 SOS and images – /images, /sos 12
- 7.3 Notifications – /notification 12
- 7.4 Referral, reports, content and admin 12

8. Key flows 14
- 8.1 Registration and email verification 14
- 8.2 Login 14
- 8.3 Forgot and reset password 14
- 8.4 SOS from the app (button, emergency screens) 15
- 8.5 Shake / chop SOS (Android native) 15
- 8.6 Voice SOS 16
- 8.7 Receiving an alert 16
- 8.8 Account deletion 16

9. Mobile app 17
- 9.1 Application shell 17
- 9.2 Navigation map 17
- 9.3 Screens 17
- 9.4 State management 18
- 9.5 Native Android modules 18
- 9.6 Permissions 19

9.7 Building and running 19

10. Deployment and operations 20

10.1 Production layout 20

10.2 Deploying the backend 20

10.3 Backups 20

10.4 Monitoring 20

11. Known issues and recommendations 21

11.1 Security 21

11.2 Functional defects 21

11.3 Recommended roadmap 22

Appendix A. Glossary 23

Appendix B. Sample backend .env 23

Appendix C. Sample SOS request 23

1. Introduction

1.1 Purpose of this document

This document is the technical reference for **Jymberee**, a personal-safety and emergency SOS platform. It explains what the product does, how the mobile app and backend are built, how data flows during an emergency, how to set up, run and deploy each part, and what risks and defects exist in the current code.

1.2 Audience

- **Developers** joining the project: architecture, code layout, API and data model.
- **DevOps / operations**: environment variables, deployment, monitoring and backups.
- **Product and QA**: features, user flows, platform differences and known issues.

1.3 Scope and source

Everything here was derived from the monorepo `unlikelycreator/Jymberee`, which contains two applications:

Folder	Application	Technology
`backend/`	REST API server	Node.js (ES modules), Express 4, MongoDB via Mongoose 8
`frontend/`	Mobile app for Android and iOS	React Native 0.86, TypeScript, custom Kotlin modules
`marketing/`	Marketing flyer and logo assets	HTML source, PNG, PDF

> **Note on accuracy**
> This document describes the code as it is, including parts that are unfinished or broken. Statements about behaviour are based on reading the source; they are not the result of running the system against production.

2. Product overview

2.1 What Jymberee does

Jymberee turns a phone into a personal safety network. When a user is in danger they trigger an SOS. The app captures their GPS location (and optionally a photo), the backend records the emergency, sends a WhatsApp alert with the address, map coordinates and photo to the user's emergency contacts, and sends a push notification to other Jymberee users in the same postal-code area, who can see the SOS on a live map.

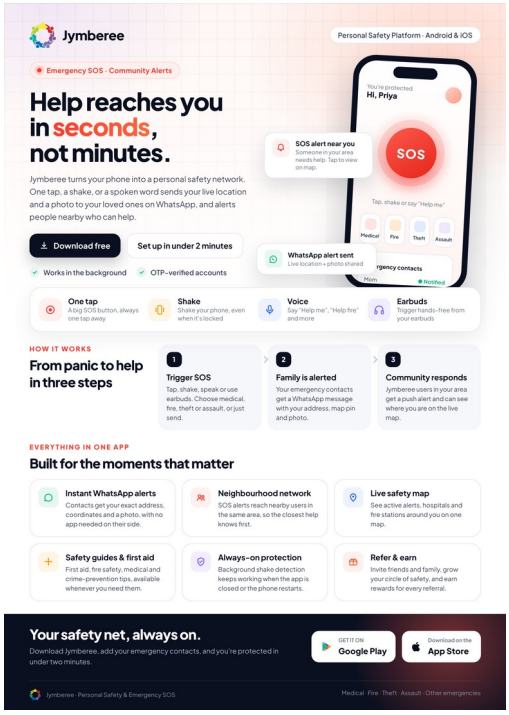


Figure 1 – Marketing flyer summarising the product

2.2 Core features

Feature	Description	Where in code
SOS by button	Bottom-tab SOS button opens a bubble menu: Fire, Theft, Rape, Medical, Other (free text). Each option takes a photo, then sends the SOS.	`navigation/TabNavigator.tsx`, `utils/sendsos.ts`
Emergency screens	Dedicated screens for Rape/Assault, Crime/Theft, Medical, Fire and Other, each with an SOS action plus safety tips and helplines.	`screens/RapeScreen.tsx`, `TheftScreen.tsx`, `Medical.tsx`, `FireScreen.tsx`, `OtherEmergencyScreen.tsx`
Voice SOS	Continuous speech recognition while the Dashboard is open. Phrases such as "help me", "help fire", "help theft", "help rape", "help medical" trigger an SOS (8 s cooldown).	`utils/voiceSos.ts`
Shake / "chop" SOS (Android)	A native foreground service reads the accelerometer and fires an SOS after 5 sharp chops within 1.3 s, even with the app in the background. Restarts after reboot.	`android/.../ShakeDetectionService.kt`, `SosDispatcher.kt`, `BootReceiver.kt`

Feature	Description	Where in code
WhatsApp alerts	Emergency contacts receive a WhatsApp template with name, phone, address, coordinates, message and photo.	`backend/controllers/v3/imageController.js`
Community alerts	Push notification to all users whose `postalCode` matches the SOS location; stored as in-app notifications (last 5 kept).	`backend/controllers/v3/notification.controller.js`
Live map	Shows the 3 most recent SOS events from other users, with profile pictures and a driving route via GraphHopper.	`screens/MapScreen.tsx`
Emergency contacts	Import from phone contacts or add manually; at least one is mandatory at registration.	`screens/EmergencyContacts.tsx`
Safety guides	First aid, safety tips, fire safety, medical safety and crime-prevention content.	`screens/FirstAidGuide.tsx` and related
Refer & earn	Each user has a referral code. Inviter earns 10, new user earns 5 (balance units). Rewarded ads on the referral screen.	`backend/services/referral.service.js`, `screens/ReferralScreen.tsx`
Ads & banners	Dashboard carousel from `/ads`; AdMob banner/rewarded ads in the app.	`backend/services/advertisement.service.js`
Account management	Profile, profile picture, preferred locations, password reset, account deletion with reason (archived).	`screens/ProfileScreen.tsx`, `DeleteAccountScreen.tsx`

2.3 SOS triggers by platform

Trigger	Android	iOS	Notes
SOS button / emergency screens	Yes	Yes	Takes a photo with the camera before sending.
Voice phrases	Yes (app open on Dashboard)	Yes (app open on Dashboard)	Listener starts on Dashboard focus; not a background service.
Shake / chop gesture	Yes (background)	No	Native Kotlin service only; no iOS equivalent.
Earbuds	Disabled	Disabled	`utils/earbudSos.ts` exists but its listener is commented out in `DashboardScreen.tsx` and `index.js`.

> **Marketing alignment**
>
> The flyer currently advertises earbud triggering and shake-while-locked as general features. In the code, earbud SOS is switched off and shake SOS is Android-only. Either enable/port these features or adjust the marketing copy before publishing.

3. System architecture

3.1 High-level view

Jymberee is a classic three-tier mobile system: a React Native client, a stateless Express REST API with MongoDB, and several third-party services for messaging, geocoding and push delivery.

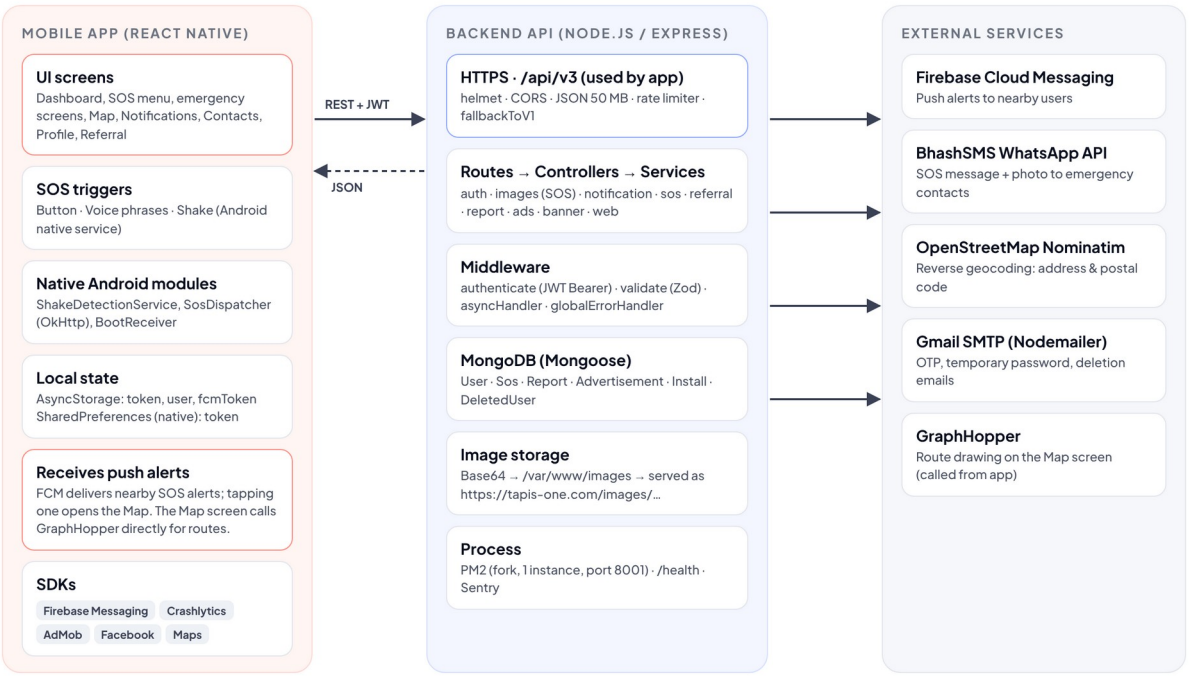


Figure 2 – System architecture

3.2 Technology stack

Layer	Technology	Version / detail
Mobile framework	React Native, React	RN ^0.86.3, React 19.2.8, TypeScript
Navigation	React Navigation	native-stack, stack, bottom-tabs v7
Mobile storage	AsyncStorage; Android SharedPreferences	Token, user JSON, FCM token
Maps & location	react-native-maps, map-clustering, get-location, geolocation-service	GraphHopper for routes
Push	@react-native-firebase/messaging, Notifee	Channel `sos_channel`
Monitoring (app)	Firebase Crashlytics	
Ads / analytics	react-native-google-mobile-ads, react-native-fbsdk-next, tracking-transparency	AdMob app IDs in `app.json`
Voice	@dev-amirzubair/react-native-voice	On-device speech recognition
Native (Android)	Kotlin, OkHttp, Google Play Services Location	Shake service, SOS dispatcher
API server	Node.js, Express 4	ES modules (`"type": "module"`)
Database	MongoDB with Mongoose 8	Database name `jimburry`

Layer	Technology	Version / detail
Validation	Zod 4	`backend/schemas/`
Auth	jsonwebtoken, bcryptjs	JWT Bearer, default expiry 7 days
Security middleware	helmet, cors, express-rate-limit	
Push (server)	firebase-admin	`sendEachForMulticast`
Email	Nodemailer (Gmail service)	OTP and transactional emails
Realtime (optional)	socket.io	Separate `socketServer.js` process
Ops	PM2, @godaddy/terminus, @sentry/node	Health checks, graceful shutdown, error tracking

3.3 External services

Service	Used for	Called from	Credentials
Firebase Cloud Messaging	Push alerts to nearby users	Backend	`backend/serviceAccountKey.json`
BhashSMS (WhatsApp)	SOS message + photo to emergency contacts (v3)	Backend	Hard-coded in `imageController.js`
dealsms.in (WhatsApp)	Text-only SOS message (legacy `/sos` route)	Backend	`INSTANCE_ID`, `WHATSAPP_ACCESS_TOKEN`
OpenStreetMap Nominatim	Reverse geocoding to address / postal code	Backend and app	None (public, rate-limited)
Gmail SMTP	OTP, temporary password, deletion emails	Backend	`EMAIL_USER`, `EMAIL_PASS`
GraphHopper	Driving route on the Map screen	App	Key hard-coded in `MapScreen.tsx`
Google AdMob, Facebook SDK	Ads, analytics	App	IDs in `app.json` / native config
MongoDB Atlas	Backup target	`backup-sync.cjs`	Hard-coded URI

3.4 Environments

Environment	API base URL	How it is selected
Production	`https://ezeelink.net/api/v3`	Default in `frontend/src/utils/api.ts`; native `SosDispatcher.kt` is hard-coded to it.
Test	`http://103.118.17.247:8001/api/v3`	Commented line in `api.ts`; swap manually.
Notification test mode	n/a	App sends `env: "test"` to `/notification/send-notification`; backend then only notifies three hard-coded test user IDs. Production code sends `env: "prod"`.

4. Repository structure

```
Jymberee/
├── README.md
├── backend/
│   ├── index.js              # entry: env, DB connect, HTTP server, /health
│   ├── app.js                # express app, middleware, /api/v1|v2|v3
│   ├── instrument.mjs        # Sentry init (loaded with --import)
│   ├── ecosystem.config.cjs  # PM2 config
│   ├── socketServer.js       # optional Socket.IO live-location server
│   ├── notify.js, backup-sync.cjs   # one-off scripts
│   ├── config/               # db.js, constants.js
│   ├── routes/v1|v2|v3/      # versioned routers
│   ├── controllers/v1|v2|v3/ # request handlers
│   ├── services/             # auth, referral, ads, email, fcm, firebase-admin
│   ├── models/               # Mongoose models
│   ├── schemas/              # Zod request schemas
│   ├── middlewares/          # auth, validate, rate limit, errors, fallback
│   └── utils/                # AppError, logger, email template, Indian states
├── frontend/
│   ├── App.tsx, index.js     # root component, SDK init
│   ├── src/navigation/       # AppNavigator, TabNavigator
│   ├── src/screens/          # ~35 screens
│   ├── src/components/       # Popup, VoiceCommandsModal, location picker
│   ├── src/context/          # Auth, Location, Sos contexts
│   ├── src/utils/            # api, sendsos, voiceSos, earbudSos, firebase
│   ├── android/              # Gradle project + Kotlin native modules
│   └── ios/                  # Xcode project, Podfile
└── marketing/                # flyer.html/.png/.pdf, assets/ (logo)
```

5. Backend

5.1 Startup sequence

1. `index.js` loads `.env` via `dotenv/config`.
2. `ConnectDB()` connects to `${LOCAL_URI}/${DB_NAME}` with up to 5 retries (1 s, 2 s, … back-off), pool size 10.
3. An HTTP server is created from `app.js` and wrapped by `@godaddy/terminus`, which adds a `/health` check (re-checks the DB) and graceful shutdown on SIGTERM.
4. The server listens on `PORT` (default 8001). Sentry is initialised before everything by `--import ./instrument.mjs`.

5.2 Request pipeline (app.js)

Order	Middleware	Purpose
1	`helmet` (CSP and COEP disabled)	Security headers
2	`morgan("combined")`	Access logging
3	`cors` with `ALLOWED_ORIGINS` (or all origins)	Cross-origin access, credentials allowed
4	`express.json` / `urlencoded` (limit 50 MB)	Large limit because SOS photos arrive as base64
5	`cookie-parser`	Cookie parsing
6	`express.static("public")`, `/uploads`	Static files
7	`authLimiter` on v1 auth paths (v3 applies it inside register, login and resend-otp handlers)	10 requests per 15 min per IP
8	`/api/v1`, `/api/v2`, `/api/v3` routers	Versioned API
9	`GET /health`	Liveness JSON
10	Catch-all 404, `globalErrorHandler`	Uniform error responses

5.3 API versioning

Three versions are mounted side by side. **The mobile app uses v3 exclusively.** v3 reuses some v2 controllers (reports) and adds banners. v3 routes are wrapped by `fallbackToV1`, which is intended to re-run a request against the v1 router if the v3 handler throws. In practice the fallback rewrites only `/api/v2` paths, so it has little effect on v3 traffic; treat v3 as the source of truth and plan to remove v1/v2 once no client uses them.

5.4 Authentication

- Tokens are JWTs signed with `ACCESS_TOKEN_SECRET`, payload `{ _id, email, phoneNumber }`, expiry `ACCESS_TOKEN_EXPIRY` (default `7d`). There is no refresh token.
- Clients send `Authorization: Bearer <token>`. The `authenticate` middleware verifies the token, loads the user (without `password` and `otp`) into `req.user`.
- Every authentication failure returns HTTP 401 with `{ success: false, message: "3" }`; the app treats message `"3"` as “log in again”.
- Passwords are hashed with bcrypt (cost 10). Accounts start as `pending` and become `active` after email OTP verification.

5.5 Validation and errors

Request bodies are validated with Zod schemas (`schemas/auth.schema.js`, `referral.schema.js`, `advertisment.schema.js`) through the `validate` middleware. Handlers are wrapped in `asyncHandler`. Business errors throw `AppError(message, statusCode)`; `globalErrorHandler` logs every error through `utils/logger.js` and returns `{ status, message }`. In production, unexpected errors return the generic message "Something went wrong".

5.6 Environment variables

Variable	Required	Used by	Description
`PORT`	No (8001)	index.js	HTTP port
`LOCAL_URI`	Yes	config/db.js	MongoDB connection string without database name
`DB_NAME`	Yes	config/db.js	Database name (production: `jimburry`)
`MONGO_URI`	For scripts	socketServer.js, notify.js	Full Mongo URI for side processes
`ACCESS_TOKEN_SECRET`	Yes	auth	JWT signing secret
`ACCESS_TOKEN_EXPIRY`	No (7d)	user.model.js	JWT lifetime
`ALLOWED_ORIGINS`	No	app.js	Comma-separated CORS origins
`EMAIL_USER`, `EMAIL_PASS`	Yes	email.service.js	Gmail account and app password
`INSTANCE_ID`, `WHATSAPP_ACCESS_TOKEN`	Legacy	controllers v1/v2/v3 sos	dealsms.in WhatsApp credentials
`FEATURE_FLAGS`	No	featureFlag.js	Comma-separated enabled flags
`SOCKET_PORT`	For socket server	socketServer.js	Socket.IO port
`NODE_ENV`	Recommended	errorHandler.js, PM2	`production` hides error details

Firebase Admin additionally reads the service account from `backend/serviceAccountKey.json` (see Section 11).

5.7 Running locally

```
cd backend
npm install
cp .env.example .env      # create it; see table above
npm run dev               # nodemon + Sentry instrument, port 8001
curl http://localhost:8001/health
```

> **Known script issue**
>
> `npm start` points to `./src/index.js`, which does not exist. Use `npm run dev` locally and PM2 (`ecosystem.config.cjs`) in production, or change the script to `./index.js`.

6. Data model

MongoDB stores most user-related data embedded in the `User` document (contacts, images, notifications, transactions). Other collections are small and flat.

6.1 User

Field	Type	Rules / notes
`name`	String	Required, min 2 chars
`email`	String	Required, unique, lower-case
`phoneNumber`	String	Unique (sparse), stored in E.164, e.g. `+919876543210`
`password`	String	bcrypt hash
`countryCode`	String	Required, e.g. `+91`
`postalCode`	String	Used to target community alerts
`fcmToken`	String | null	Firebase device token; cleared when FCM reports it invalid
`profilePicture`	String	Public image URL
`preferredLocations[]`	{street, city, state, postalCode, country}	Optional saved addresses
`emergencyContacts[]`	{name, number, countryCode}	At least one required; numbers normalised to E.164
`images[]`	{url, uploadedAt}	Every SOS photo URL is appended
`notifications[]`	{title, message, senderId, sos[], latitude, longitude, mark_as_read, createdAt}	Only the latest 5 are kept
`otp`	String	6-digit email OTP; cleared on verification
`status`	`pending` | `active`	Active after OTP verification
`referralCode`	String	Unique 6-char code, upper-case
`referrer`, `referredUsers[]`	ObjectId refs	Referral graph
`balance`	Number	Referral earnings, min 0
`transactions[]`	{title, amount, type: credit|debit, relatedUser, date}	Earnings ledger
`createdAt`, `updatedAt`	Date	Mongoose timestamps

6.2 Other collections

Model	Fields	Purpose
`Sos`	`userId`, `latitude`, `longitude`, `description`, `timestamp`	One active SOS per user (previous one is deleted on each new SOS)
`DeletedUser`	Copy of user fields + `userId`, `reason`, `deletedAt`	Archive written before an account is deleted
`Report`	`userId`, `reportType` (fire|crime), `details`, `timestamp`	User-submitted incident reports
`Advertisement`	`title`, `imageUrl`, `redirectUrl`, `active`	Dashboard ads and banners (fallback ads returned if empty)
`Install`	`ip`, `installId`, `platform`, `userAgent`, `installedAt`	Install tracking from the login screen
`Notification`	`userId`, `title`, `message`, `senderId`, `sos`, `read`	Defined but not used; notifications live inside `User`
`EmergencyContacts`	Legacy model	Not used by v3

7. API reference (v3)

Base URL: `https://ezeelink.net/api/v3`. All bodies are JSON. “Auth” means a valid `Authorization: Bearer <JWT>` header is required.

7.1 Auth – /auth

Method & path	Auth	Request body	Result
POST `/register`	No	`name, email, password (≥6), phoneNumber, countryCode, postalCode?, preferredLocations?, emergencyContacts[{name, number, countryCode?}] (≥1), referralCode?`	Creates `pending` user, applies referral bonus, emails OTP
POST `/verify-otp`	No	`email? | phoneNumber?, otp (6 digits)`	Activates user; returns `{ user, token }`
POST `/resend-otp`	No	`email`	Generates and emails a new OTP
POST `/login`	No	`phoneNumber (E.164), password`	`{ user, token }`; if not active, re-sends OTP and returns 403
POST `/forgot-password`	No	`email? | phoneNumber?`	Sets a random 6-digit temporary password and emails it
DELETE `/cancel-account`	No	`email`	Deletes an unfinished registration
GET `/get-fcm`	No*	–	Returns stored FCM token (*see defects: needs auth)
POST `/reset-password`	Yes	`password (≥6)`	Changes password
POST `/save-token`	Yes	`fcmToken | null`	Stores device push token
POST `/profile-picture`	Yes	`imageUrl`	Sets profile picture URL
PUT `/update-profile`	Yes	`name, phoneNumber (E.164), email, postalCode?, preferredLocations?`	Updates profile
PUT `/emergency-contacts`	Yes	`name, number`	Adds an emergency contact
DELETE `/emergency-contacts/:id`	Yes	–	Removes a contact
DELETE `/delete-account`	Yes	`reason`	Archives to `DeletedUser`, emails confirmation, deletes user and referral links

7.2 SOS and images – /images, /sos

Method & path	Auth	Request body	Result
POST `/images/upload-image`	Yes	`image (data:image/...;base64 | null), latitude, longitude, description`	Main SOS endpoint (see 8.4). Returns `{ message, imageUrl, sos }`. 429 if an SOS is already processing for this user.
GET `/images/images`	Yes	–	User's uploaded SOS images
POST `/images/profilepic`	Yes	`image (base64)`	Uploads and sets profile picture
POST `/sos`	Yes	`latitude, longitude, description`	Legacy SOS: text WhatsApp via dealsms.in and push to **all** users
GET `/sos/active`	Yes	–	3 most recent SOS from other users, with sender profile

7.3 Notifications – /notification

Method & path	Auth	Request body	Result
POST `/send-notification`	Yes	`title, message?, latitude, longitude, description, env ("prod" | "test")`	Pushes to users in the same postal code; `{ success, successCount }`
GET `/get-notifications`	Yes	–	Latest 5 notifications with sender and SOS populated
DELETE `/delete-notification/:notificationId`	Yes	–	Removes one
DELETE `/delete-all-notifications`	Yes	–	Clears all
PATCH `/mark-as-read/:notificationId`	Yes	–	Sets `mark_as_read = 1`

7.4 Referral, reports, content and admin

Method & path	Auth	Request body	Result
POST `/referral/generate`	Yes	–	Issues a new referral code
POST `/referral/invite`	Yes	–	Returns the user's code for sharing
POST `/referral/apply`	Yes	`referralCode`	Inviter +10, user +5 (once per user, no self-referral)
GET `/referral/earnings`	Yes	–	`{ balance, transactions }`
GET `/referral/invited`	Yes	–	Invited users (name, email, phone)
POST `/report/crime-report`	Yes	`details`	Creates crime report (*currently broken, see 11.2)

Method & path	Auth	Request body	Result
POST `/report/fire-report`	Yes	`details`	Creates fire report
GET `/ads`	No	`?limit`	Active advertisements (fallbacks if none)
GET `/banner/banners`	No	`?limit`	Banners (*currently broken, see 11.2)
GET `/web/users`	No	–	All users without password/OTP (**unauthenticated**)
GET `/web/users/:id`	No	–	User name by ID
POST `/web/install`	No	`installId?, platform?`	Upserts install record
GET `/web/installs`	No	–	All install records
GET `/health` (root)	No	–	`{ status: "ok", timestamp }`

8. Key flows

8.1 Registration and email verification

1. User fills the multi-step Register screen: name, email, phone with country, password, postal code / location, at least one emergency contact, optional referral code.
2. App calls `POST /auth/register`. The backend normalises the phone and every contact number to E.164, rejects duplicates, hashes the password, generates a referral code and creates the user with status `pending`.
3. If a valid referral code was supplied, the inviter receives 10 and the new user 5 balance units, recorded as transactions.
4. A 6-digit OTP is saved on the user and emailed. The app shows the OTP screen; the user may resend or cancel (`DELETE /auth/cancel-account`).
5. `POST /auth/verify-otp` activates the account and returns a JWT. The app stores token and user in AsyncStorage, saves the token in native SharedPreferences (for the shake service), and registers the FCM token with `POST /auth/save-token`.

8.2 Login

`POST /auth/login` with phone number and password. If the account is still `pending`, the server emails a fresh OTP and returns 403 so the app can switch to OTP entry. On success the app stores the token, records the install (`POST /web/install`) and navigates to the Home tabs.

8.3 Forgot and reset password

`POST /auth/forgot-password` replaces the password with a random 6-digit temporary password and emails it. The user then logs in and sets a new password from the profile (`POST /auth/reset-password`).

8.4 SOS from the app (button, emergency screens)

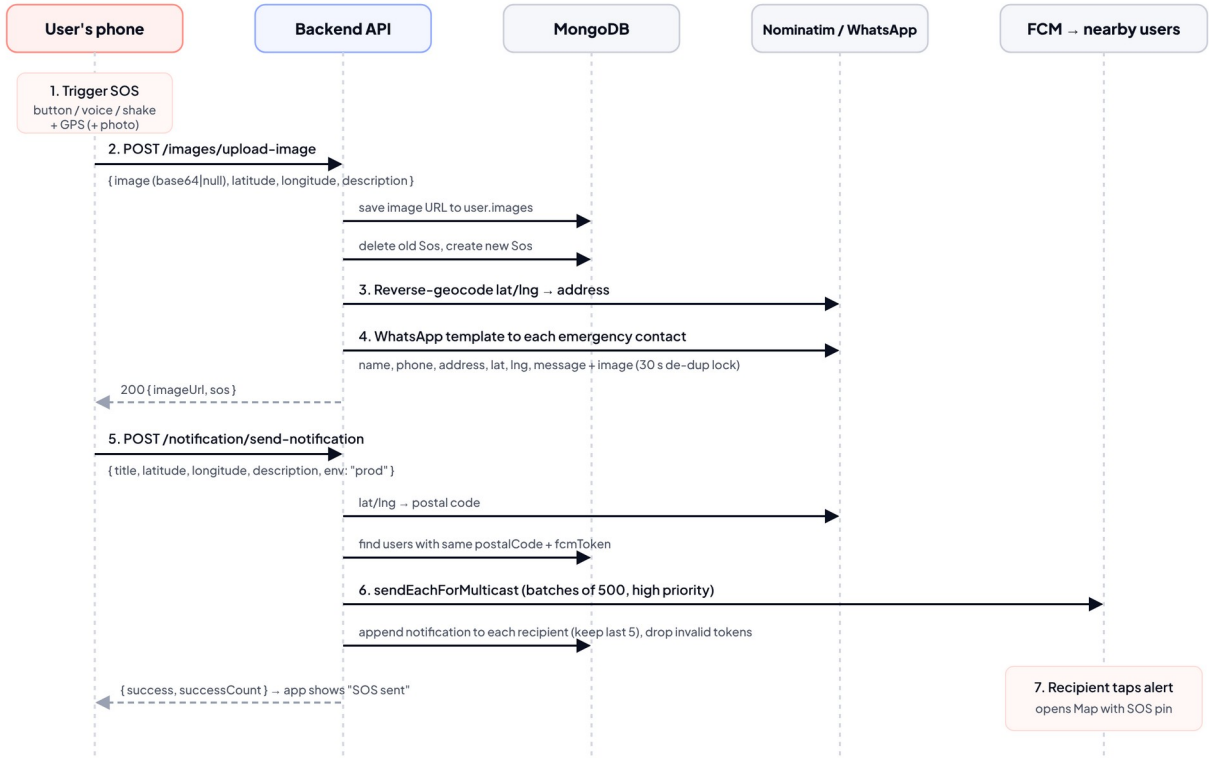


Figure 3 – SOS sequence (in-app trigger)

1. The app checks that at least one emergency contact exists, requests camera permission, gets a high-accuracy GPS fix (60 s timeout) and opens the camera.
2. A global "Sending SOS…" overlay appears (`SOS_SENDING` event). The photo is sent as base64 to `POST /images/upload-image` with coordinates and description.
3. The backend takes a per-user 30 s lock, writes the image to `/var/www/images/jym-<userId>-<timestamp>.<ext>`, appends its public URL to `user.images`, replaces the user's `Sos` document and reverse-geocodes the address.
4. A WhatsApp template (`sos_alert_final`) with name, phone, address, latitude, longitude, message and the image is sent to each unique emergency contact through BhashSMS (second 30 s lock prevents duplicates).
5. The app then calls `POST /notification/send-notification`. The backend resolves the postal code of the SOS location, finds other users with that postal code and an FCM token, sends high-priority pushes in batches of 500, removes invalid tokens and appends the notification to each recipient (keeping 5).
6. The app shows "SOS sent successfully" (`SOS_SENT`) or an error (`SOS_FAILED`).

8.5 Shake / chop SOS (Android native)

- `DashboardScreen` calls `ShakeModule.startService()` which starts `ShakeDetectionService` as a foreground service with a persistent notification. `BootReceiver` restarts it after reboot.
- Accelerometer data is low-pass filtered; a "chop" is a direction flip above 15.5 m/s². Five chops within 1.3 s trigger an SOS, followed by a 2 s cooldown.
- `SosDispatcher` reads the token from SharedPreferences, gets the last known location, and calls `upload-image` (with `image: null`, so a default image URL is used) and then `send-notification` directly over HTTPS with OkHttp. No photo is taken.

8.6 Voice SOS

While the Dashboard is focused, `startVoiceSosListener()` runs continuous speech recognition with automatic restarts. Recognised text is normalised and matched against phrases (`help help help`, `help me`, `help fire`, `help theft`, `help rape`, `help medical`, …). A match sends the SOS through the same two API calls, with an 8 s cooldown between triggers. iOS permission strings mention “help” and “bachao”.

8.7 Receiving an alert

`MyFirebaseMessagingService` (Android) and `utils/firebase.ts` handle incoming pushes on channel `sos_channel`. Tapping a notification navigates to the Map screen, which loads `GET /sos/active` and shows the latest SOS pins, sender photos and a route from the viewer to the SOS location. The Notifications screen lists stored alerts and supports delete and delete-all.

8.8 Account deletion

`DELETE /auth/delete-account` with a reason: a confirmation email is sent, the full user document is copied to `DeletedUser`, the user is deleted and removed from any referrer’s `referredUsers`.

9. Mobile app

9.1 Application shell

`index.js` initialises the Facebook SDK and registers `App`. `App.tsx` wraps the app in `GestureHandlerRootView`, `SafeAreaProvider`, `LocationProvider` and `AuthProvider`, sets up FCM (permission on Android 13+, token stored locally and refreshed), requests iOS tracking permission and renders global SOS loading and result modals driven by `DeviceEventEmitter` events.

9.2 Navigation map

Navigator	Routes
Auth stack (logged out)	`Login`, `Register`, `ForgotPassword`
Home tabs	`Dashboard`, `SOS` (centre button opening the SOS bubble menu), `Map`
App stack	`Profile`, `EmergencyContacts`, `ResetPassword`, `ProfileResetPassword`, `rapescreen`, `CrimeScreen`, `Medical`, `FireScreen`, `EmergencyScreen`, `Other`, `first-aid-guide`, `safety-tips`, `fire-safety-tips`, `medical-safety-tips`, `crime-prevention-tips`, `ReferralScreen`, `Privacy`, `DeleteAccount`, `PersonalInfo`, `Location`, `Map`

9.3 Screens

Screen	Purpose	API calls
LoginScreen	Phone + password login, OTP verification for pending accounts, install tracking	`/auth/login`, `/auth/verify-otp`, `/auth/resend-otp`, `/web/install`
RegisterScreen	Multi-step sign-up incl. contacts import, location and referral code	`/auth/register`, `/auth/verify-otp`, `/auth/resend-otp`, `/auth/cancel-account`
ForgotPasswordScreen	Request temporary password	`/auth/forgot-password`
DashboardScreen	Home: ads carousel, emergency category cards, safety guides, referral entry, notifications, starts voice and shake listeners	`/ads`, Nominatim
TabNavigator (SOS menu)	Animated SOS bubbles: Fire, Theft, Rape, Medical, Other	via `sendsos.ts`
Rape / Theft / Medical / Fire / Other screens	Category SOS action, helplines and tips	via `sendsos.ts`
MapScreen	Live SOS pins, profile images, clustering, GraphHopper route	`/sos/active`, GraphHopper
Notification	List, delete, delete-all alerts	`/notification/*`
EmergencyContacts	Add, import, delete contacts	`/auth/emergency-contacts`

Screen	Purpose	API calls
ProfileScreen, PersonalInfoScreen, LocationScreen	Profile data, picture, preferred locations	`/auth/update-profile`, `/images/profilepic`
ResetPassword	Change password	`/auth/reset-password`
ReferralScreen	Code, invite share, apply code, earnings, invited users, rewarded ads	`/referral/*`
PrivacyScreen	Privacy policy and EULA links (ezeelink.net)	–
DeleteAccountScreen	Delete with reason	`/auth/delete-account`
FirstAidGuide, SafetyTips, FireSafetyTips, MedicalSafetyTips, CrimePreventionTips	Static guidance content	–
OfflinePage	Shown without connectivity	–
DoctorCallScreen, SecurityScreen, Camera	Prototype screens with placeholder data	–

9.4 State management

- **AuthContext**: `user`, `emergencyContacts`, `isLoggedIn`, `isLoading`; hydrated from AsyncStorage keys `isLoggedIn` and `user` on start.
- **LocationContext**: current map region (default San Francisco coordinates until a fix is obtained).
- **SosContext**: list of SOS locations (provider defined, lightly used).
- **AsyncStorage keys**: `token`, `user`, `isLoggedIn`, `fcmToken`, `tempUserEmail`.
- The axios instance in `utils/api.ts` attaches the token from AsyncStorage to every request.

9.5 Native Android modules

Class	Responsibility
`ShakeModule` / `ShakePackage`	React Native bridge: `startService()`, `stopService()`, `saveToken(token)`; forwards shake broadcasts to JS
`ShakeDetectionService`	Foreground service with accelerometer chop detection
`SosDispatcher`	Sends SOS over HTTPS from native code (hard-coded production URLs)
`LocationHelper`	Fused location helper
`BootReceiver`	Restarts the shake service after device boot
`MyFirebaseMessagingService`	Receives FCM messages and token refreshes
`MainActivity`, `MainApplication`	App entry, intent handling, package registration

9.6 Permissions

Permission	Platform	Why
Fine / coarse location, foreground service (location)	Android, iOS (when in use / always)	SOS coordinates, map
Camera	Android, iOS	SOS photo, profile picture
Record audio, speech recognition	Android, iOS	Voice SOS
Read / write contacts	Android, iOS	Import emergency contacts
Post notifications	Android 13+	SOS alerts
Foreground service (sensor), wake lock, boot completed	Android	Background shake detection
Bluetooth connect	Android	Earbud integration (currently disabled)
Photo library	iOS	Profile picture
App tracking transparency	iOS	Ads personalisation

9.7 Building and running

```
cd frontend
npm install
npm start                       # Metro bundler
npm run android                 # debug build on device/emulator
cd ios && pod install && cd .. && npm run ios

# Android release (requires keystore, not in git)
cd android && ./gradlew assembleRelease   # or bundleRelease for Play Store
```

Release signing reads `MYAPP_UPLOAD_STORE_FILE`, `MYAPP_UPLOAD_KEY_ALIAS`, `MYAPP_UPLOAD_STORE_PASSWORD` and `MYAPP_UPLOAD_KEY_PASSWORD` from `android/gradle.properties`; the file `jymberee-release-key.keystore` must be placed in `android/app/` by hand. Firebase config files `google-services.json` and `GoogleService-Info.plist` are committed.

10. Deployment and operations

10.1 Production layout

Item	Value
Public API	`https://ezeelink.net/api/v3` (reverse proxy to the Node process)
Process manager	PM2 app `jymberee-production-8001`, fork mode, 1 instance, port 8001, restart at 400 MB, Sentry preloaded
Image directory	`/var/www/images` on the API host, served publicly as `https://tapis-one.com/images/<file>`
Database	MongoDB (local instance, database `jimburry`); backup copy on MongoDB Atlas
Health	`GET /health` (terminus checks DB connectivity)

10.2 Deploying the backend

```
git pull
cd backend && npm ci
pm2 start ecosystem.config.cjs     # first time
pm2 reload jymberee-production-8001 # subsequent deploys
pm2 logs jymberee-production-8001
```

10.3 Backups

`backup-sync.cjs` is a Windows-oriented script that runs `mongodump` of the local database to `C:\backup` and `mongorestore --drop` into MongoDB Atlas. It is run manually; there is no scheduled job in the repository. Because it drops the target before restoring, a failed dump followed by a restore could wipe the Atlas copy; schedule it with care and keep dated snapshots.

10.4 Monitoring

- Sentry (backend) via `instrument.mjs`; Firebase Crashlytics (app).
- `morgan` access logs and `utils/logger.js` error logs on the server.
- Recommended: uptime probe on `/health`, alert on PM2 restarts, and dashboards for WhatsApp and FCM failure counts.

11. Known issues and recommendations

The following were found while reviewing the code. Severity reflects impact if exploited or triggered in production.

11.1 Security

#	Severity	Issue	Recommendation
S1	Critical	`GET /web/users` and `/web/installs` have no authentication and return every user's email, phone, emergency contacts, locations, images and notifications.	Remove from public routing or protect with an admin role; never return contacts or locations in bulk.
S2	Critical	Secrets committed to git: Firebase service account (`serviceAccountKey.json`), MongoDB Atlas password (`backup-sync.cjs`), BhashSMS credentials (`imageController.js`), GraphHopper key (`MapScreen.tsx`), keystore password in `gradle.properties`.	Rotate every credential, move them to environment variables / secret store, purge from git history.
S3	High	Email OTP never expires (`otpExpires` is set in code but is not in the schema, so it is never saved or checked). `verify-otp` and `forgot-password` have no rate limit (register, login and resend-otp do), so a 6-digit OTP can be brute-forced.	Add `otpExpires` to the schema and check it; apply `authLimiter` to verify-otp and forgot-password; lock after N failed attempts.
S4	High	Forgot-password resets the real password immediately and anyone who knows an email or phone can trigger it, locking the owner out.	Use a time-limited reset token/OTP and only change the password after it is confirmed.
S5	Medium	`DELETE /auth/cancel-account` deletes by email without authentication.	Allow only for `pending` accounts and require the OTP or a signed registration token.
S6	Medium	CORS defaults to all origins with credentials; Socket.IO allows `*`.	Set `ALLOWED_ORIGINS` explicitly in production.
S7	Medium	iOS `NSAppTransportSecurity` allows insecure HTTP for an exception domain; test API uses plain HTTP.	Use HTTPS for test servers and remove the ATS exception.
S8	Low	JWT lifetime 7 days with no refresh or revocation.	Shorter access tokens plus refresh tokens; invalidate on password change.

11.2 Functional defects

#	Area	Defect	Effect
D1	Reports	`createCrimeReport` (v2 controller, used by v3) reads `details` without destructuring it from `req.body`.	Every crime report returns HTTP 500.

#	Area	Defect	Effect
D2	Banners	`GetBanners` calls `getActiveAdvertisements(limit)` but the service expects `(req, limit)` and reads `req.baseUrl`.	`GET /banner/banners` throws and returns an error.
D3	Auth	`GET /auth/get-fcm` is mounted before `authenticate` but reads `req.user._id`.	Always fails; also should require auth.
D4	Contacts	App calls `PATCH /auth/emergency-contacts/toggle`, which does not exist on the server.	Toggle action in Emergency Contacts fails.
D5	Notifications	Community alerts depend on an exact `postalCode` string match from Nominatim; users who did not enter a postal code, or whose code is formatted differently, never receive alerts. If Nominatim is slow or down, no one is notified.	Missed alerts. Prefer geospatial queries (store last known location with a 2dsphere index and query by radius).
D6	SOS	SOS sending blocks on the camera; if the user cancels the camera, no SOS is sent at all.	Send location-only SOS immediately; attach the photo as a follow-up.
D7	SOS	WhatsApp sends to contacts run sequentially inside the request, each with a 20 s timeout.	Slow responses for users with many contacts; move to a background queue.
D8	SOS	In-memory locks (`sendingSOS`, `whatsappLock`) only work with a single process.	Use Redis or DB-based locks before scaling beyond one PM2 instance.
D9	Backend	`npm start` points to a missing file; `form-data` is imported but not a direct dependency; `app.js` creates upload folders one directory above the backend.	Startup and packaging fragility.
D10	App	API base URL and `env: "prod"/"test"` are switched by editing code; native code duplicates the production URL.	Use build-time config (e.g. react-native-config) for both JS and native.
D11	App	Earbud SOS code is present but disabled; shake SOS is Android-only.	Marketing and store listing must match.
D12	App	Leftover files: `LoginScreen copy.tsx`, `ios/true`, prototype screens with example.com data.	Remove to reduce confusion.

11.3 Recommended roadmap

1. **Immediately**: rotate and remove leaked secrets; lock down `/web/*`; fix D1–D4.
2. **Short term**: OTP expiry and rate limits on v3; safe password reset; location-only SOS without camera; environment-based config.
3. **Medium term**: geospatial alert targeting, background job queue for WhatsApp/FCM, retire v1/v2 routes, automated tests and CI.
4. **Product**: decide on earbud and iOS background triggers; align marketing claims.

Appendix A. Glossary

Term	Meaning
SOS	An emergency alert raised by a user
Emergency contact	A person (phone number) who receives the WhatsApp alert
Community alert	Push notification sent to other Jymberee users in the same postal area
FCM	Firebase Cloud Messaging, Google's push notification service
OTP	One-time password, 6 digits, sent by email to verify an account
E.164	International phone format, e.g. `+919876543210`
Chop gesture	Rapid back-and-forth phone movement detected by the Android shake service
PM2	Node.js process manager used in production

Appendix B. Sample backend .env

```
PORT=8001
NODE_ENV=production
LOCAL_URI=mongodb://localhost:27017
DB_NAME=jimburry
MONGO_URI=mongodb://localhost:27017/jimburry
ACCESS_TOKEN_SECRET=<long-random-string>
ACCESS_TOKEN_EXPIRY=7d
ALLOWED_ORIGINS=https://ezeelink.net
EMAIL_USER=<gmail-address>
EMAIL_PASS=<gmail-app-password>
INSTANCE_ID=<dealsms-instance>
WHATSAPP_ACCESS_TOKEN=<dealsms-token>
FEATURE_FLAGS=
SOCKET_PORT=8002
```

Appendix C. Sample SOS request

```
POST /api/v3/images/upload-image
Authorization: Bearer <jwt>
Content-Type: application/json

{
  "image": "...",
  "latitude": 19.0760,
  "longitude": 72.8777,
  "description": "Medical Emergency Reported"
}

200 OK
{ "message": "SOS created successfully", "imageUrl":
"https://tapis-one.com/images/jym-...jpeg", "sos": { ... } }
```